



Faculty of Women, Ain Shams University

GRADUATION PROJECT

Interactive Calculus Explorer

A Web-Based Mathematical Analysis Tool

Prepared by

Abeer Abdelhady Abdeltwab

Malak Ahmed Mohamed

Supervised by

Dr. Azza Soliman

Mathematics Department

Faculty of Women, Ain Shams University

May 2026

Table of Contents

Chapter 1: Introduction and Background	1
1.1 Abstract	1
1.2 Introduction	1
1.2.1 Background	1
1.2.2 Problem Statement	2
1.2.3 Project Objectives	2
1.2.4 Scope	3
1.3 Literature Review	3
1.3.1 Computer Algebra Systems	3
1.3.2 Web-Based Mathematical Visualization	3
1.3.3 Security in Mathematical Parsing	4
1.3.4 Numerical Methods for Root Finding	4
Chapter 2: System Design and Implementation	5
2.1 System Design and Architecture	5
2.1.1 Architecture Overview	5
2.1.2 Component Design	5
2.2 Technology Stack	6
2.2.1 Justification of Choices	8
2.3 Implementation Details	8
2.3.1 Code Structure	8
2.3.2 Core Functions	9
2.3.3 Safe Parsing Implementation	10
2.3.4 Integration Strategy	11
2.3.5 Critical Point Detection	11
2.3.6 Max/Min Classification	12
Chapter 3: Testing, Results, and Discussion	13
3.1 Testing and Validation	13
3.1.1 Automated Test Suite	13
3.1.2 Manual Test Cases	15
3.1.3 Key Test Results	17
3.2 Results and Discussion	18
3.2.1 Achieved Objectives	18
3.2.2 Performance	19
3.2.3 Limitations	19
Chapter 4: Future Work, Conclusion, References, and Appendices	20
4.1 Future Work	20
4.2 Conclusion	21
4.3 References	21
4.4 Appendix A: Installation and Usage	22
4.4.1 Prerequisites	22
4.4.2 Installation	22
4.4.3 Running the Application	23
4.4.4 Running Tests	23
4.5 Appendix B: Supported Functions	24

Chapter 1: Introduction and Background

1.1 Abstract

This graduation project presents the design and implementation of an Interactive Calculus Explorer, a web-based mathematical analysis application built using Python and modern scientific computing libraries. The application enables users to input arbitrary mathematical functions and automatically produces comprehensive analytical results including function plots, first derivatives with slope evaluation, definite integration with symbolic and numeric fallback methods, critical point detection, and local maxima/minima classification using the second derivative test and sign-change analysis.

The system prioritizes security by employing safe expression parsing through SymPy's controlled parser, completely avoiding the use of `eval()` or `exec()`. The application features a responsive Streamlit-based user interface with real-time computation and interactive parameter adjustment. Extensive testing validates correctness across polynomial, trigonometric, exponential, and rational functions, including robust handling of discontinuities and domain restrictions.

1.2 Introduction

1.2.1 Background

Calculus forms the mathematical foundation of modern science and engineering. The ability to analyze functions — their behavior, rates of change, accumulated quantities, and extreme values — is essential across disciplines from physics to economics. Traditional approaches rely on manual computation or expensive commercial software such as MATLAB and Mathematica, creating barriers for students and educators.

Recent advances in open-source scientific computing, particularly the SymPy computer algebra system and the Streamlit web framework, have made it feasible to build powerful yet accessible mathematical tools. This project leverages these technologies to create an interactive calculus analysis platform that is free, web-based, and safe for educational use.

1.2.2 Problem Statement

Students and educators frequently need tools to visualize and analyze mathematical functions for learning and teaching purposes. Existing solutions present several limitations: commercial tools (MATLAB, Wolfram Alpha Pro) require expensive licenses; general-purpose graphing calculators lack symbolic computation capabilities; web-based tools often rely on unsafe evaluation methods (`eval/exec`) that pose security risks; and many available tools do not provide comprehensive analysis combining plotting, differentiation, integration, and critical point classification in a single unified interface.

1.2.3 Project Objectives

The primary objectives of this project are:

- Develop a web-based application for comprehensive mathematical function analysis
- Implement safe expression parsing without `eval()` or `exec()` to prevent code injection attacks
- Provide symbolic and numeric computation of derivatives, integrals, and critical points
- Create an intuitive, interactive user interface with real-time parameter adjustment
- Handle mathematical edge cases including discontinuities, complex values, and divergent integrals

- Validate the system through comprehensive automated and manual testing

1.2.4 Scope

The project focuses on single-variable real-valued functions of x . It supports standard mathematical functions (trigonometric, exponential, logarithmic, polynomial, and rational) and provides first-derivative analysis. The application handles functions over user-specified intervals with configurable plot resolution. While the system does not extend to multivariable calculus or differential equations, the modular architecture allows future expansion.

1.3 Literature Review

1.3.1 Computer Algebra Systems

Computer Algebra Systems (CAS) perform symbolic mathematical operations including simplification, differentiation, integration, and equation solving. Commercial systems like Mathematica (Wolfram Research, 1988) and Maple (Maplesoft, 1985) pioneered this field. The open-source SymPy library, created by Ondřej Čertík in 2007, provides comparable symbolic computation capabilities in Python. SymPy has been validated in numerous academic publications and supports safe expression parsing through its `sympy.parsing` module.

1.3.2 Web-Based Mathematical Visualization

Desmos and GeoGebra represent popular web-based graphing tools that provide interactive function visualization. However, these tools prioritize graphing over comprehensive calculus analysis. Streamlit, an open-source Python framework launched in 2019, enables rapid development of data-focused web applications. Its reactive programming model makes it particularly suitable for scientific computing applications where parameter changes should immediately trigger recomputation.

1.3.3 Security in Mathematical Parsing

Input sanitization is a critical concern in applications that evaluate user-supplied mathematical expressions. The use of Python's `eval()` or `exec()` functions creates severe security vulnerabilities including arbitrary code execution. Research in secure parsing recommends using abstract syntax tree (AST) validation or dedicated parsing libraries. SymPy's `parse_expr` with controlled transformations and restricted local dictionaries provides a well-established approach to safe mathematical expression evaluation.

1.3.4 Numerical Methods for Root Finding

Critical point detection requires finding roots of derivative functions. The Brent's method (1973) combines bisection, secant, and inverse quadratic interpolation for reliable convergence. SciPy's implementation (`scipy.optimize.brentq`) provides a robust fallback when symbolic solutions are unavailable. The hybrid approach of attempting symbolic solutions first with numeric fallback ensures both accuracy and robustness.

Chapter 2: System Design and Implementation

2.1 System Design and Architecture

2.1.1 Architecture Overview

The application follows a modular two-layer architecture separating the mathematical computation engine from the user interface layer. This separation enables independent testing, maintenance, and potential reuse of the math engine in other contexts.

The system consists of two Python modules: `math_engine.py` (the computation core) and `app.py` (the Streamlit user interface). The math engine exposes eight primary functions that handle the complete analysis pipeline from parsing through classification.

2.1.2 Component Design

2.1.2.1 Parsing Layer

The parsing layer converts user-supplied text strings into SymPy symbolic expressions. It uses `sympy.parsing.sympy_parser.parse_expr` with the `convert_xor` and `implicit_multiplication_application` transformations. A controlled local dictionary restricts allowed symbols to `x` and permitted functions (`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `exp`, `log`, `sqrt`, `Abs`, `pi`, `E`). After parsing, a validation step checks for unknown symbols or function calls and raises a descriptive `ParseError` if any are found.

2.1.2.2 Numeric Evaluation Layer

The numeric layer converts symbolic expressions to efficient NumPy-vectorized callables using SymPy's `lambdify` function. The `safe_eval` wrapper handles three categories of invalid values: complex numbers (from operations like `sqrt` of negatives) are masked to NaN with a user warning; infinite values (from division by zero) are masked to NaN to prevent plot distortion; and fully invalid ranges produce NaN arrays with diagnostic warnings.

2.1.2.3 Analysis Layer

The analysis layer computes derivatives (symbolic differentiation via `sympy.diff`), integrals (symbolic integration with numeric fallback via SciPy or mpmath), and critical points (hybrid symbolic-numeric root finding with Brent's method refinement). The classification module applies the second derivative

test for definitive results and falls back to first-derivative sign-change analysis when the second derivative test is inconclusive.

2.1.2.4 Presentation Layer

The Streamlit interface provides a sidebar for input parameters and a main panel for results. The sidebar contains the function input, plot range controls, integration interval, derivative toggle, and point-slope evaluation. The main panel displays the Matplotlib plot (with optional derivative overlay and color-coded critical point markers), LaTeX-rendered symbolic expressions, slope values as metric cards, integral results with method attribution, and a critical points table with classification details.

2.2 Technology Stack

The project utilizes the following technologies, chosen for their maturity, community support, and interoperability:

Component	Technology	Version	Role
User Interface	Streamlit	≥ 1.28	Web application framework with reactive widgets
Symbolic Math	SymPy	≥ 1.12	Computer algebra: parsing, differentiation, integration, solving
Numeric Comp.	NumPy	≥ 1.24	Vectorized array operations for plotting and evaluation
Plotting	Matplotlib	≥ 3.7	Publication-quality 2D plot generation
Numeric Fallback	SciPy	≥ 1.11	Numeric integration (quad) and root finding (brentq)
Language	Python	≥ 3.10	Core programming language

2.2.1 Justification of Choices

Streamlit was selected over alternatives (Flask, Django, Dash) for its minimal boilerplate, built-in reactivity, native LaTeX rendering, and Matplotlib integration. SymPy provides the only production-quality open-source CAS in Python with safe parsing capabilities. NumPy and SciPy form the standard scientific Python stack, ensuring broad compatibility and well-documented behavior. Matplotlib was chosen for its stability and the ability to generate figures as in-memory objects for Streamlit's `st.pyplot`.

2.3 Implementation Details

2.3.1 Code Structure

The codebase is organized into three files with clear responsibilities:

File	Lines	Purpose
math_engine.py	~300	All mathematical logic: 8 public functions + helpers
app.py	~180	Streamlit UI: sidebar inputs, layout, display logic
requirements.txt	5	Dependency specification for pip installation

2.3.2 Core Functions

The math engine exposes eight primary functions that form the analysis pipeline:

Function	Signature	Description
parse_function	$(\text{expr_str}) \rightarrow \text{Expr}$	Safe string-to-expression parsing with validation
make_numeric	$(\text{expr}) \rightarrow \text{callable}$	SymPy expression to NumPy-vectorized function
safe_eval	$(\text{func}, \text{xs}) \rightarrow \text{ndarray}, \text{list}$	NaN/inf/complex-safe array evaluation
compute_derivative	$(f) \rightarrow \text{Expr}$	Symbolic first derivative with simplification
slope_at	$(f', x_0) \rightarrow \text{float} \text{None}$	Point slope evaluation with error handling
compute_integral	$(f, a, b) \rightarrow \text{tuple}$	Definite integral: symbolic $\rightarrow$ SciPy $\rightarrow$ mpmath fallback
find_critical_points	$(f, f', \text{xmin}, \text{xmax}) \rightarrow \text{list}$	Hybrid symbolic + numeric root detection
classify_points	$(f, f', f'', \text{pts}) \rightarrow \text{list}$	2nd derivative test + sign-change fallback

2.3.3 Safe Parsing Implementation

The parsing system is the first line of defense against code injection. It operates in three stages:

- Stage 1 — Controlled Parsing: The input string is processed by `parse_expr` with a restricted `local_dict` containing only the symbol `x` and twelve allowed functions/constants. The `convert_xor` transformation enables the caret (^) operator as an alias for exponentiation.
- Stage 2 — Symbol Validation: After parsing, `free_symbols` is inspected to ensure no unknown variables appear. Any symbol other than `x` triggers a descriptive `ParseError`.
- Stage 3 — Function Validation: All Function atoms in the expression tree are checked against the allowed list. Unknown functions (e.g., user-attempted `os.system` calls) are rejected.

This three-stage approach ensures that even if the parser produces a valid SymPy expression, it will be rejected if it references anything outside the allowed mathematical vocabulary.

2.3.4 Integration Strategy

The integration module implements a three-tier fallback strategy to maximize both accuracy and reliability:

- Tier 1 — Symbolic Integration: SymPy's `integrate` function attempts to find a closed-form antiderivative and evaluate the definite integral. This produces exact results when possible (e.g., $\int x^2 dx = x^3/3$).
- Tier 2 — SciPy Numeric Quadrature: If symbolic integration fails or returns an unevaluated expression, SciPy's adaptive Gauss-Kronrod quadrature (`scipy.integrate.quad`) provides high-accuracy numeric results with error estimation.
- Tier 3 — mpmath Quadrature: As a final fallback, mpmath's arbitrary-precision quadrature handles cases where both SymPy and SciPy fail. If all three tiers fail, the system reports the integral as divergent or unconverged.

2.3.5 Critical Point Detection

Critical point detection uses a hybrid approach combining symbolic and numeric methods:

- Symbolic Phase: SymPy's `solveset` attempts to solve $f'(x) = 0$ over the given interval. When this yields a finite set of solutions, they are collected directly.
- Numeric Phase: A fine grid (2000 points) of $f'(x)$ values is evaluated. Sign changes between adjacent grid points indicate bracketed roots, which are refined using Brent's method (or bisection as fallback).

- Singularity Detection: The derivative's denominator (for rational expressions) is analyzed to find points where f' is undefined but f is defined — these are also critical points.
- Deduplication: All discovered points are sorted and deduplicated with a tolerance of 10^{-6} to remove near-duplicates from different detection methods.

2.3.6 Max/Min Classification

Classification proceeds through a two-stage test:

- Second Derivative Test: For each critical point c , $f''(c)$ is evaluated. If $f''(c) < 0$, the point is a local maximum; if $f''(c) > 0$, it is a local minimum.
- Sign-Change Fallback: When $f''(c) \approx 0$ or is undefined, the sign of f' is sampled at $c - \epsilon$ and $c + \epsilon$. A sign change from positive to negative indicates a maximum; negative to positive indicates a minimum; no change suggests an inflection point.

2.4. Security Considerations

2.4.1 Input Sanitization

The most critical security requirement is preventing arbitrary code execution through user-supplied mathematical expressions. The application enforces this through multiple layers:

- No `eval()` or `exec()`: These Python built-ins are completely absent from the codebase. All expression evaluation passes through SymPy's safe parsing pipeline.
- Restricted Symbol Space: Only the variable `x` and a fixed set of mathematical functions are permitted. Attempts to use Python built-ins (`import`, `open`, `os`, `sys`) are blocked.
- Expression Tree Validation: Even valid SymPy expressions are post-validated to ensure no unexpected symbols or functions appear.

2.4.2 Error Handling

All mathematical operations are wrapped in `try/except` blocks to prevent application crashes from malformed input. Errors are displayed as friendly Streamlit error messages rather than raw Python tracebacks. Numeric evaluation masks invalid results (NaN, infinity, complex values) rather than propagating exceptions.

2.4.3 Denial of Service Mitigation

While the application does not implement explicit rate limiting, several design choices mitigate computational abuse: plot resolution is capped via the UI slider; integration uses adaptive methods that converge efficiently; and the Streamlit framework itself provides session isolation between users.

Chapter 3: Testing, Results, and Discussion

3.1 Testing and Validation

3.1.1 Automated Test Suite

The project includes a comprehensive automated test suite (`test_math_engine.py`) containing 111 individual test assertions organized into eight test groups:

Test Group	Tests	Coverage
1. Safe Parsing	29	23 valid expressions + 6 rejected malicious inputs
2. Numeric Evaluation	10	x^2 , $1/x$ discontinuity, $\sqrt{x}$ complex, $\log(x)$, $\tan(x)$
3. Derivatives & Slopes	10	Polynomial, trig, chain rule, product rule, undefined slopes
4. Definite Integration	9	Symbolic integrals, negative area, divergent integrals
5. Critical Points	16	Cubic, sine, x^2 , $x^4 - 2x^2$, constants, singularities
6. Classification	15	2nd derivative test, sign-change fallback, inflection points
7. Plot Generation	9	8 function types + derivative toggle (no-crash tests)
8. End-to-End Scenarios	13	Three spec scenarios fully validated

All 111 tests pass successfully, providing confidence in the correctness and robustness of the mathematical engine.

3.1.2 Manual Test Cases

In addition to automated tests, 28 manual test cases are documented across six groups for human verification of the complete application including UI behavior:

Group	Cases	Focus
Core Mathematics	4	Cubic, sine, reciprocal, divergent integral
Parsing Features	3	Caret notation, implicit multiplication, trig identity
Function Types	6	Exponential, sqrt, abs, log, higher-degree, tangent
Specific Features	5	Point slope, rational functions, inflection, constants, inverse trig
Error Handling	6	Unknown vars/functions, empty input, bad ranges, injection
UI/UX	4	Derivative toggle, resolution, narrow zoom, wide range

3.1.3 Key Test Results

Test Case 1: Cubic Polynomial

Function: $x^3 - 6x^2 + 9x + 1$ over $[-1, 5]$. The system correctly identifies two critical points at $x = 1$ (local maximum, $f(1) = 5$) and $x = 3$ (local minimum, $f(3) = 1$), both classified via the second derivative test. The definite integral evaluates to exactly 18.0 using symbolic methods.

Test Case 2: Sine Function

Function: $\sin(x)$ over $[-2\pi, 2\pi]$. Four critical points are correctly identified at $x \approx -4.71, -1.57, 1.57$, and 4.71 , with alternating max/min classification matching the expected behavior of the sine function.

Test Case 3: Reciprocal (Discontinuity)

Function: $1/x$ over $[-5, 5]$. The application handles the discontinuity at $x = 0$ without crashing, correctly masking infinite values and producing a plot with two separate branches. The integral over $[1, 5]$ correctly evaluates to $\ln(5) \approx 1.6094$.

3.2 Results and Discussion

3.2.1 Achieved Objectives

All primary objectives have been successfully met:

- The web application provides comprehensive function analysis through a single unified interface
- Expression parsing is entirely safe, with no use of `eval()` or `exec()`
- Symbolic and numeric computation work together through intelligent fallback mechanisms
- The Streamlit interface provides real-time interactivity with minimal latency
- Edge cases including discontinuities, complex domains, and divergent integrals are handled gracefully
- 111 automated tests and 28 manual test cases validate system correctness

3.2.2 Performance

The application delivers responsive performance for typical use cases. Function plotting with 1000 points completes in under 100 milliseconds. Symbolic differentiation and integration are effectively instantaneous for standard functions. Critical point detection, which combines symbolic solving and numeric grid search, typically completes within 200 milliseconds. The primary performance bottleneck occurs with very wide plot ranges (e.g., $[-100, 100]$ for trigonometric functions) where the critical point table may grow large.

3.2.3 Limitations

The current implementation has several known limitations:

- Single-variable functions only (no multivariable calculus or parametric curves)
- First derivative analysis only (higher-order derivatives are computed internally but not exposed)
- The constant function $f(x) = c$ has $f'(x) = 0$ everywhere, which could generate numerous spurious critical points
- No persistent state between sessions (each page load starts fresh)
- Piecewise functions are not directly supported through the text input

Chapter 4: Future Work, Conclusion, References, and Appendices

4.1 Future Work

Several enhancements could extend the project's capabilities:

- Second and Higher Derivatives: Expose $f''(x)$ and beyond for concavity analysis and inflection point detection
- Parametric and Polar Plots: Support $x(t)$, $y(t)$ parametric curves and $r(\theta)$ polar functions
- Taylor Series: Display Taylor/Maclaurin series expansion around user-specified points
- Differential Equations: Basic ODE solving and direction field visualization
- Export Options: Allow users to download plots as PNG/SVG and results as PDF or CSV
- Mobile Responsiveness: Optimize the Streamlit layout for smartphone and tablet use
- Collaborative Features: Shared sessions for classroom use with real-time synchronization
- GPU Acceleration: Use CuPy for numeric evaluation on large datasets

4.2 Conclusion

This graduation project has successfully delivered an Interactive Calculus Explorer — a comprehensive, secure, and user-friendly web application for mathematical function analysis. The system demonstrates that modern open-source tools (SymPy, NumPy, Streamlit) can produce professional-quality mathematical software without relying on expensive commercial platforms.

The application's key strengths lie in its layered security model (safe parsing without eval/exec), its hybrid computation approach (symbolic with numeric fallback), and its robust handling of mathematical edge cases. With 111 automated tests and 28 documented manual test cases, the system provides strong confidence in its correctness and reliability.

The modular architecture ensures that the mathematical engine can be reused in other contexts (API services, Jupyter notebooks, command-line tools), while the Streamlit interface provides immediate accessibility to students and educators without any installation or configuration beyond pip install.

4.3 References

- [1] Meurer, A. et al., “SymPy: Symbolic Computing in Python,” PeerJ Computer Science, vol. 3, e103, 2017.
- [2] Harris, C.R. et al., “Array Programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.
- [3] Hunter, J.D., “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.
- [4] Virtanen, P. et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” Nature Methods, vol. 17, pp. 261–272, 2020.
- [5] Brent, R.P., “Algorithms for Minimization Without Derivatives,” Prentice-Hall, 1973.
- [6] Streamlit Inc., “Streamlit Documentation,” <https://docs.streamlit.io>, 2024.
- [7] OWASP Foundation, “Code Injection,” OWASP Web Security Testing Guide, 2023.
- [8] Desmos Inc., “Desmos Graphing Calculator,” <https://www.desmos.com>, 2024.

4.4 Appendix A: Installation and Usage

4.4.1 Prerequisites

- Python 3.10 or higher
- pip (Python package manager)

4.4.2 Installation

Clone or download the project files (app.py, math_engine.py, requirements.txt) to a directory. Then install dependencies:

```
pip install -r requirements.txt
```

The requirements.txt file contains:

```
streamlit>=1.28.0  
sympy>=1.12  
numpy>=1.24  
matplotlib>=3.7  
scipy>=1.11
```

4.4.3 Running the Application

Launch the application with the Streamlit CLI:

```
streamlit run app.py
```

The application will open automatically in your default web browser at <http://localhost:8501>.

4.4.4 Running Tests

Execute the automated test suite:

```
python test_math_engine.py
```

Expected output: 111 passed / 0 failed / 111 total.

4.5 Appendix B: Supported Functions

Function	Syntax	Example	Description
Sine	$\sin(x)$	$\sin(2 \cdot x)$	Trigonometric sine
Cosine	$\cos(x)$	$\cos(x)^{**2}$	Trigonometric cosine
Tangent	$\tan(x)$	$\tan(x/2)$	Trigonometric tangent
Arcsine	$\text{asin}(x)$	$\text{asin}(x/2)$	Inverse sine
Arccosine	$\text{acos}(x)$	$\text{acos}(x/3)$	Inverse cosine
Arctangent	$\text{atan}(x)$	$\text{atan}(2 \cdot x)$	Inverse tangent
Exponential	$\exp(x)$	$\exp(-x^{**2})$	e raised to the power x
Natural Log	$\log(x)$	$\log(x+1)$	Natural logarithm (base e)
Square Root	$\text{sqrt}(x)$	$\text{sqrt}(x^{**2}+1)$	Principal square root
Absolute Value	$\text{Abs}(x)$ or $\text{abs}(x)$	$\text{abs}(x-2)$	Absolute value
Pi	pi	$\sin(\text{pi} \cdot x)$	Mathematical constant π
Euler's Number	E or e	E^{**x}	Mathematical constant $e \approx 2.718$
Power (caret)	x^n	$x^3 + x^2$	Alias for x^{**n}